

Topic 3C: Learning and Generalization

Last updated: September 29, 2026

Surbhi Goel

Acknowledgements: The automata examples draw on Liu et al. [4, 2]; the regression discussion on Garg et al. [5], von Oswald et al. [6], and Hill et al. [9]; and the length-generalization discussion on Veitsman et al. [1], Yang and Chiang [3], and Huang et al. [11].

AI Disclosure: These notes were drafted and revised with AI assistance. If you find an error, please let me know.

A Transformer can represent the correct function and still learn a rule that fails on new inputs. We study two kinds of generalization:

- **Fixed-size generalization:** Does the model handle shifts in the input distribution even when length is fixed?
- **Length generalization:** Even if the model is correct on every short input, what supports extension to longer ones?

We will work through several toy tasks and see how generalization can fail, what causes it, and how it can be fixed.

1 Remembering the Most Recent Write

The *flip-flop* task maintains a one-bit memory. A write w stores the following bit, an ignore i leaves the memory unchanged, and a read r asks for the stored bit. For example,

$$w0 \ i1 \ w1 \ i0 \ i0 \ r? \longrightarrow 1.$$

Instructions and bits occupy separate positions. As in induction heads, the answer follows an earlier token: here, the most recent write.

Changing the instruction distribution. Liu et al. [2] train at token length $T = 512$ and test new instruction frequencies:

Distribution	Ignore	Write	Read
Training	0.80	0.10	0.10
Sparse test	0.98	0.01	0.01
Dense test	0.10	0.45	0.45

The first instruction is a write and the last is a read; these are the probabilities for the remaining instructions. Sparse streams have long gaps between writes: 98% of sampled instructions are ignores. Dense streams have many competing writes. Models that perform well on the training distribution make errors in both cases.

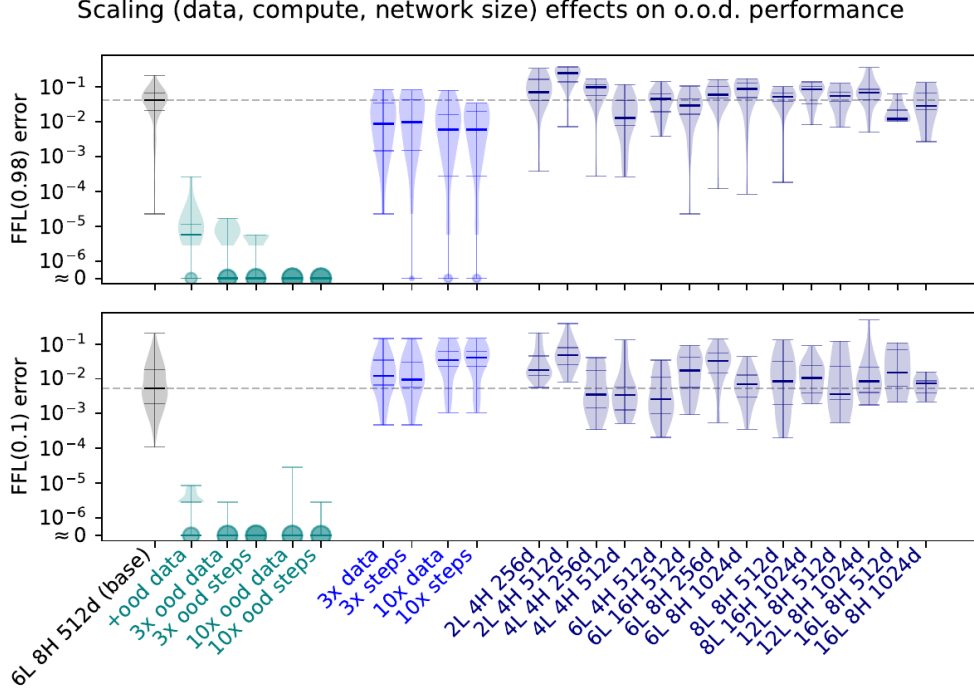


Figure 1: Flip-flop interventions at length 512: sparse errors above, dense errors below. Teal groups broaden the training distribution; blue groups increase data, steps, or model size. Error axes are logarithmic; zero-error runs appear at the bottom. From Liu et al. [2, Figure 9, left].

What can go wrong? Consider a read head whose candidate values $b_j \in \{0, 1\}$ carry bits from earlier positions. It forms

$$z = \sum_j \alpha_j b_j, \quad \alpha_j = \frac{e^{s_j}}{\sum_k e^{s_k}},$$

and returns one when $z > 1/2$. The following scores are illustrative; we normalize over the candidates shown.

Wrong selection. Suppose the last write stores one, but a recent ignore carries zero. A learned preference for nearby positions can outweigh the preference for writes when the write-read gap is large. If the last write has score one and the ignore has score two, then

$$z = \frac{e}{e + e^2} = \frac{1}{1 + e} \approx 0.269.$$

The read returns zero. Multiplying both scores by a large positive number makes attention concentrate even more on the wrong position.

Mixing values. Even if the last write has the highest score, its attention weight need not exceed one half. Suppose the head attends only to writes: the last write stores one with score two, and m earlier writes store zero with score one each. Then

$$z = \frac{e^2}{e^2 + me} = \frac{e}{e + m}.$$

For $m = 2$, $z \approx 0.576$ and the read is correct. For $m = 3$, $z \approx 0.475$ and it is wrong. Replacing ignores with writes can change this mixture while keeping the total token length fixed.

These examples explain why accuracy on ordinary streams need not control sparse gaps or dense competing writes. Liu et al. [2, Sections 4.1 and 5.3, Appendix C.2] analyze related mechanisms in restricted models and demonstrate wrong selections in simplified flip-flop experiments. They do not identify one circuit used by every trained model.

What helps? Training on a mixture of the original, sparse, and dense distributions reduces both kinds of error. Six of twenty-five runs have no detected errors in the reported tests (Figure 1). More data or training steps can improve sparse errors while worsening dense errors; changing model size gives no consistent improvement across both tests.

2 Parity at Unusual Counts

The parity task asks the model to predict whether the number of ones seen so far is odd or even at each position. Start from state zero, flip the state on a 1, and leave it unchanged on a 0.¹ For example, 1, 0, 1, 1 produces the states 1, 1, 0, 1. At position t , the target is

$$C_t = \sum_{j=1}^t x_j, \quad y_t = C_t \bmod 2.$$

A possible solution. Liu et al. [4] suggest that the model may count the ones in the prefix and decode the count as odd or even. Uniform attention supplies the fraction C_t/t ; at a known position t , the count can be recovered from that fraction. A ReLU MLP can then supply the alternating labels. For inputs of length at most T , writing $[u]_+ = \max\{u, 0\}$,

$$g(c) = c + 2 \sum_{k=1}^{T-1} (-1)^k [c - k]_+$$

equals $c \bmod 2$ at every integer $c \in \{0, \dots, T\}$. Each ReLU introduces a bend at one count, changing the slope so that the function rises from zero to one, then falls back to zero. This is a valid finite-range decoder. Training still has to constrain its values at the counts used for testing. In contrast, majority needs only a threshold on the fraction of ones.

Testing atypical counts. Liu et al. [4] train on length-40 strings of independent fair bits, supervising the parity of every prefix. They then fix the length and choose the number of ones before placing them at random positions. Accuracy falls at atypical counts (Figure 2).

What can go wrong? At each position t , the training count satisfies

$$C_t \sim \text{Binomial}(t, 1/2), \quad \mathbb{E}[C_t] = t/2, \quad \text{sd}(C_t) = \sqrt{t}/2.$$

Most training prefixes at position t have counts near $t/2$. A predictor can fit these count–position pairs and fail on rare ones.

For example, count three is common around position six, but at position forty it has probability

$$\Pr(C_{40} = 3) = 2^{-40} \binom{40}{3} \approx 9 \times 10^{-9}.$$

¹Parity and flip-flop are useful toy tasks as they are base instances of broader finite-state tasks (see Liu et al., *Transformers Learn Shortcuts to Automata* [4]).

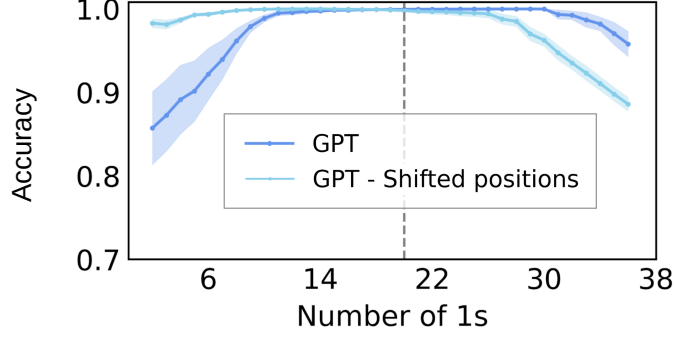


Figure 2: Parity accuracy at length 40 as the number of ones changes. The lighter curve uses random starting position indices during training. From Liu et al. [4, Figure 6, right].

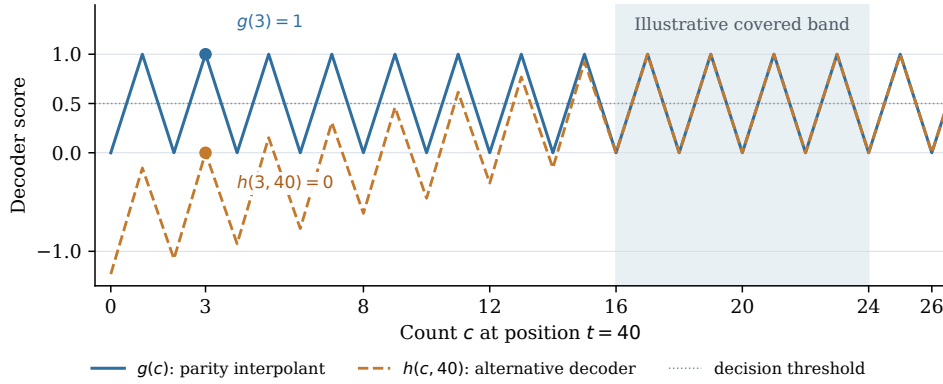


Figure 3: Illustrative ReLU decoder scores at position 40. Both match parity on counts 16–24, but disagree at count three. The shaded band illustrates coverage; it is not the support of the paper’s training distribution. These are constructed scores, not measurements of the learned model.

Errors on these late, low-count prefixes contribute almost nothing to average training loss. Seeing count three early and position forty separately does not constrain their combination.

Two decoders that agree on common inputs. The decoder can use position as well as count. To make the ambiguity explicit, set $T = 40$ and compare g with another ReLU MLP score,

$$h(c, t) = g(c) - \frac{1}{13}[t/2 - 4 - c]_+.$$

A score above $1/2$ predicts odd parity. At $t = 40$, the extra term vanishes for $c \geq 16$, but subtracts one at $c = 3$. They agree whenever $|c - t/2| \leq 4$, an illustrative band around the mean count. Yet

$$h(3, 6) = g(3) = 1, \quad h(3, 40) = g(3) - \frac{13}{13} = 0.$$

Thus h gets count three right early and wrong late. Both scores are correct on the band; correctness there does not determine their behavior outside it (Figure 3). The fair-bit distribution has nonzero probability outside this band, but late, very low counts receive little training weight.

If the decoder depended *only* on the exact count, learning count three early would also determine its answer late. The failure above is possible because the representation or decoder can depend on position.

Random offsets in the supplied position indices improve low-count accuracy [4, Appendix B.2.3], supporting this explanation. Offsets weaken the association with the absolute position index; the distribution of C_t at prefix length t remains unchanged. Sampling counts explicitly at each length would give rare counts more weight during training.

3 Linear Regression in Context

Let’s look at a more complex in-context learning task: implementing a machine learning algorithm in-context. In particular, given examples in the context coming from some linear function, predict the label of the next example.

Choose $w_\star \in \mathbb{R}^d$ and labels $y_i = w_\star^\top x_i$. The prompt is

$$P = (x_1, y_1, \dots, x_T, y_T, x_\star), \quad y_\star = w_\star^\top x_\star.$$

Here T counts examples; the alternating prompt has $2T + 1$ positions. Training minimizes squared prediction error over many independently sampled tasks and inputs. At evaluation the Transformer’s parameters stay fixed: it must infer the current function from the prompt. We take $w_\star, x_1, \dots, x_T, x_\star$ independently from $\mathcal{N}(0, I_d)$, with no label noise. Each prompt has a different $w_\star$.

The least-squares benchmark. Let X have rows $x_i^\top$ and let $y = (y_1, \dots, y_T)^\top$. Minimum-norm least squares chooses

$$\hat{w}_{\text{LS}} \in \arg \min_w \|Xw - y\|_2^2, \quad \hat{y}_{\text{LS}} = x_\star^\top \hat{w}_{\text{LS}},$$

using the minimizer of smallest norm when it is not unique. The labels determine the component of $w_\star$ in the span of the examples. Under the Gaussian prior, the remaining component has conditional mean zero. Thus least squares gives the conditional mean prediction, which minimizes squared loss. Each independent example determines one more direction, giving normalized error

$$\frac{1}{d} \mathbb{E}[(\hat{y}_{\text{LS}} - y_\star)^2] = \max\{1 - T/d, 0\}.$$

At $T = d$, the examples determine the function almost surely. Garg et al. [5] train with $d = 20$ and find that a Transformer approaches this benchmark on new prompts from the training distribution (Figure 4).

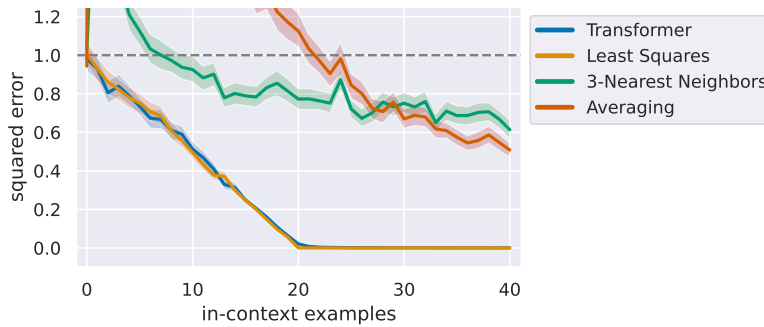


Figure 4: Prediction error divided by d on fresh Gaussian regression prompts. The trained Transformer approaches least squares. From Garg et al. [5, Figure 2].

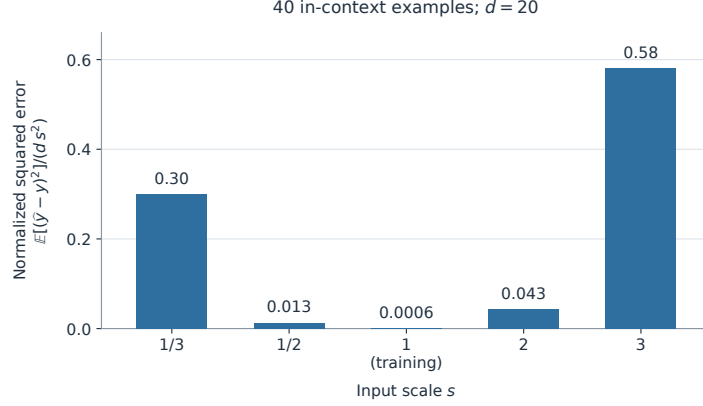


Figure 5: Scale shift at $d = 20$, $T = 40$. The model trains at scale one. Error is divided by ds^2 ; least squares remains exact. Plot of numbers reported by Garg et al. [5, Section 4].

An attention update. For the squared loss,

$$L(w) = \frac{1}{2T} \sum_j (w^\top x_j - y_j)^2, \quad w^{(1)} = \frac{\eta}{T} \sum_j y_j x_j$$

is one gradient step from zero. Its query prediction is

$$\hat{y}_{\text{GD},\eta} = \frac{\eta}{T} \sum_j (x_*^\top x_j) y_j.$$

This is *linear attention*: set $q_* = x_*$, $k_j = x_j$, and $v_j = y_j$, divide the weighted sum by T , and multiply the output by η [6]. There is no softmax; the weights may be negative. For alternating input and label tokens, a preceding layer associates each x_j with its label, as in the induction construction.

One step does not solve least squares in general. Additional layers can implement further updates using residuals $y_j - x_j^\top w^{(\ell)}$; richer constructions implement more elaborate regression procedures [7, 8].

Changing the scale of a prompt. Keep T and w_* fixed, and multiply every input, including the query, by $s > 0$. The labels also scale by s :

$$P_s = (sx_1, sy_1, \dots, sx_T, sy_T, sx_*), \quad y_{*,s} = sy_*.$$

Least squares keeps the same coefficient estimate because $\|(sX)w - sy\|^2 = s^2\|Xw - y\|^2$. Its prediction therefore obeys

$$\hat{y}_{\text{LS}}(P_s) = s\hat{y}_{\text{LS}}(P).$$

Garg et al. test this shift with $d = 20$ and $T = 40$. Although the examples still determine the function, the model makes larger relative errors away from its training scale. Dividing the error by ds^2 accounts for the change in target magnitude (Figure 5).

For a fixed step size η , the one-step predictor scales as

$$\hat{y}_{\text{GD},\eta}(P_s) = \frac{\eta}{T} \sum_j ((sx_*)^\top (sx_j))(sy_j) = s^3 \hat{y}_{\text{GD},\eta}(P).$$

The target scales linearly; this predictor scales cubically. Replacing η by η/s^2 restores linear scaling for this predictor.

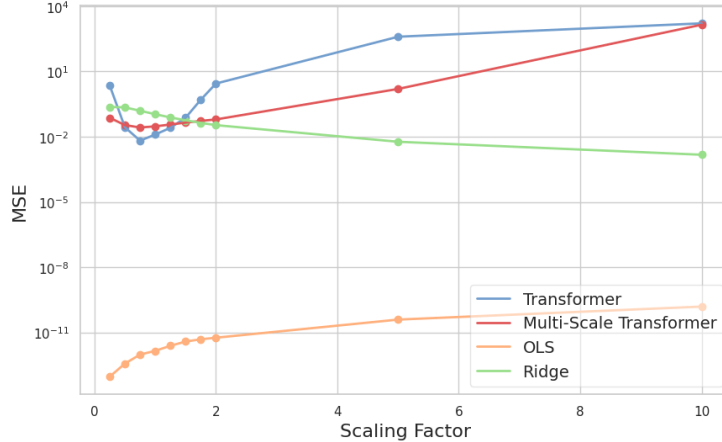


Figure 6: Transformers trained at one scale or multiple scales. This plot uses raw MSE, unlike Figure 5. OLS is ordinary least squares. From Hill et al. [9, Figure 9].

Broader scale coverage. Hill et al. [9] compare the same architecture trained at scale one or at scales chosen from $\{1, 2, 3\}$. The broader training distribution improves performance across scales (Figure 6). It does not enforce the exact scaling law, especially beyond the training range.

Further reading for fixed-size generalization. Garg et al. [5] also study covariance shifts, and Zhang et al. [10] analyze covariance sensitivity in a single-layer linear-attention model. Liu et al. [4] study recurrent scratchpads and finite-state computations; Akyürek et al. [7] and Fu et al. [8] study richer regression algorithms. The companion [notebook](#) separates familiar-length shifts from longer-input tests and compares position encodings in small exploratory runs.

4 Generalization to Longer Inputs

Suppose a model is correct on *every* short input, including rare counts and unusual sequences. When does correctness extend to longer inputs?

Three retrieval rules. Recall the induction head task. In the prompt

$$a b \quad \cdots \quad a c \quad \cdots \quad a ?,$$

the final a is the query and the omitted tokens contain no a . An earlier occurrence is eligible only if its following token lies strictly before the query. *Most-recent retrieval* returns c , as in our induction head example. *First-match retrieval* returns b . In *unique-match retrieval*, there is exactly one eligible earlier occurrence, and we copy its following token. First and most-recent retrieval assume at least one eligible occurrence.

Veitsman et al. [1] fine-tune GPT-2 1.5B separately on these tasks with lengths at most 100, then test on lengths 101–200. Earlier query occurrences lie in the first half of the test context, forcing a longer retrieval gap. Training uses random position offsets to expose more position indices. Among runs that fit their task, first and unique retrieval remain perfect on the measured test; most-recent retrieval makes errors (Figure 7). The evaluation uses sampled inputs; perfect measured accuracy does not establish correctness on all longer inputs.

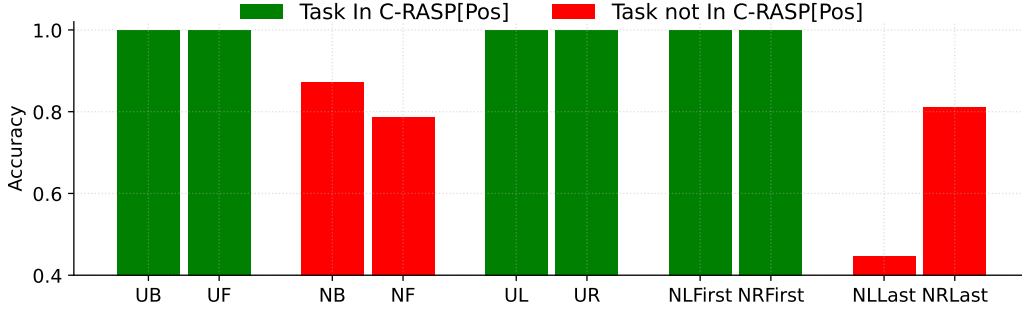


Figure 7: Length-shift accuracy. The relevant tasks are UR (unique), NRFirst (first), and NRLast (most recent); each copies a continuation. Other bars concern different copying tasks. The axis starts at 0.4. Colors indicate membership in C-RASP[pos], the counting-program class defined in Section 5.1. From Veitsman et al. [1, Figure 6].

5 Length Generalization and C-RASP

Huang et al. [11] give a length-generalization guarantee for tasks expressible in C-RASP[pos], under suitable assumptions on training coverage and model selection. The result uses a formal Transformer model and an idealized learner that fits all short inputs and prefers simpler models. This is a sufficient condition. It does not guarantee ordinary training, and it does not rule out generalization for tasks outside this class.

5.1 C-RASP: Counting Programs

C-RASP describes a computation as a fixed list of instructions. Each instruction defines a new value at every input position. There are two types of values: Boolean indicators and integers, including counts [3, 11].

What is available at each step? At position i , keep the values defined by the earlier instructions. Later instructions can reuse *both* the indicators and the integers. A count can be added to another count, compared with a constant, or used in a conditional. It need not first be reduced to a Boolean value. These are program steps; one step need not correspond to one Transformer layer.

Allowed operations. Write P, R for Boolean sequences and C, D for integer sequences. All operations below use sequences defined by earlier instructions.

Operation	New value at position i
Token test	$Q_a(i) = [x_i = a]$
Boolean logic	$\top, \neg P(i), P(i) \wedge R(i)$
Prefix count	$C(i) = \#_{j \leq i} P(j)$
Integer arithmetic	$1, C(i) + D(i), C(i) - D(i)$
Conditional	$P(i) ? C(i) : D(i)$
Comparison	$C(i) \leq D(i)$

Brackets denote a Boolean test, and $\top$ is always true. A conditional chooses $C(i)$ when $P(i)$ is true and $D(i)$ otherwise. Other fixed integer constants, $\vee$, $=$, and $>$ can be built from these operations.

Arithmetic and comparisons use values at the same position i . The prefix count combines information across positions: it counts where an earlier Boolean sequence $P(j)$ is true. This instruction does not directly sum an integer sequence across positions.

Example: marking the first a . The following three instructions produce an indicator, a count, and another indicator:

$$Q_a(i) = [x_i = a], \quad C_a(i) = \#_{j \leq i} Q_a(j), \quad F_a(i) = Q_a(i) \wedge [C_a(i) = 1].$$

Input x_i	a	b	a	c	a
Q_a (Boolean)	1	0	1	0	1
C_a (integer)	1	1	2	2	3
F_a (Boolean)	1	0	0	0	0

When computing F_a , both earlier rows are available. At b , the count is still one, but $Q_a = 0$, so $F_a = 0$. After this instruction, later instructions can use all three rows.

The number of instructions stays fixed as the input grows, although the counts can grow. A line cannot refer to its own output at the preceding position, so a recurrent update such as $y_i = y_{i-1} \text{ XOR } x_i$ is not an allowed instruction. For a yes/no task, we read the final Boolean value at position T .

C-RASP[pos]: fixed position tests. For retrieval, we need to mark the token immediately after an occurrence. The position extension allows the fixed relation $j = i - 1$ inside a count. It gives a one-position shift of a Boolean sequence:

$$\text{Prev}(P)(i) = [\#_{j \leq i, j=i-1} P(j) > 0].$$

For example,

Position	1	2	3	4	5
P	1	0	1	0	1
$\text{Prev}(P)$	0	1	0	1	0

At the first position there is no predecessor, so the result is false. The offset stays one at every input length.

More generally, the extension permits local relations that depend only on $i - j$ and are false beyond a fixed distance. It also allows fixed periodic position tests, such as $[i \bmod 2 = 0]$. The distance bounds and periods do not grow with the input. Full prefix counts remain available. Here [pos] describes permitted program operations, not a particular position encoding. Huang et al. call this class C-RASP[periodic, local] [11, 1].

5.2 Which Retrieval Rules Can We Write?

Fix a finite alphabet. For each query symbol a and candidate answer b , compute the following values at every position. We prepare the same instructions for every pair of symbols; the number of copies is fixed because the alphabet is fixed.

Unique-match retrieval. In $abxxa$, the final a is the query, and x differs from a and b . Mark each completed ab pair at its second token, then count marked pairs before the query:

$$P_{ab}(i) = \text{Prev}(Q_a)(i) \wedge Q_b(i),$$

$$N_{ab}(i) = \#_{j < i} P_{ab}(j).$$

The two new rows are

Input	a	b	x	x	a (query)
P_{ab} (Boolean)	0	1	0	0	0
N_{ab} (integer)	0	0	1	1	1

At the query, $N_{ab} = 1$, so we return b . The unique-match promise makes the answer unambiguous. The strict count $j < i$ excludes the query itself as a continuation.² For a general query, use the counts corresponding to its current token: output b when $\bigvee_a (Q_a(i) \wedge [N_{ab}(i) > 0])$ holds. This is a fixed C-RASP[pos] program [1, Appendix A.2, Lemma 4].

First-match retrieval. Use the same computation, but mark only the first occurrence of a . We already computed its indicator $F_a(i) = Q_a(i) \wedge [C_a(i) = 1]$. Replace the pair test by

$$P_{ab}(i) = \text{Prev}(F_a)(i) \wedge Q_b(i),$$

and keep the count N_{ab} and output rule unchanged. On $abac a$, F_a marks only the first position. The shift therefore marks b but not c . At the final query, $N_{ab} = 1$ and $N_{ac} = 0$, so the answer is b . Every step uses an allowed operation [1, Appendix A.2, Lemma 5]. For both tasks, a fixed default handles inputs outside the task’s promise; counts also test whether an eligible match exists or is unique.

Most-recent retrieval. On the same input $abac a$, the most-recent rule returns c . No fixed C-RASP[pos] program computes this rule at all input lengths. Veitsman et al. [1, Appendix A.2, Lemma 6] give the formal proof.

5.3 Interpreting the Class

Parity is also outside C-RASP[pos] [11, Lemma 11]. Counting the ones is allowed. A fixed periodic test on the position index, such as $[i \bmod 2 = 0]$, is also allowed. Determining whether an arbitrary input-dependent count is odd or even is a different computation. The earlier ReLU decoder handled a bounded count range by adding a hinge at each count; it did not give one fixed program for every input length. The formal nonexpressibility result rules out other C-RASP[pos] implementations too.

The classification concerns one fixed program working at all lengths:

Task	C-RASP[pos]	Construction or result
Unique retrieval	Yes	Mark matches, shift, count pairs
First retrieval	Yes	Mark the first match, then reuse the program
Most-recent retrieval	No	Nonexpressibility result
Parity	No	Nonexpressibility result

²Strict counts use the same operations: $\#_{j < i} P(j) = \#_{j \leq i} P(j) - (P(i) ? 1 : 0)$.

The first two retrieval rules fall within the class covered by the conditional length-generalization result. The most-recent rule and parity do not receive a guarantee from this sufficient condition. This distinction does not assert that every model trained on a task inside the class will generalize, or that tasks outside it cannot generalize.

References

- [1] Yana Veitsman, Mayank Jobanputra, Yash Sarrof, Aleksandra Bakalova, Vera Demberg, Ellie Pavlick, and Michael Hahn. [Born a Transformer—Always a Transformer?](#) arXiv:2505.21785v2, 2025. The comparisons cited here use this version’s Sections 3 and 4.3.
- [2] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. [Exposing Attention Glitches with Flip-Flop Language Modeling](#). NeurIPS, 2023.
- [3] Andy Yang and David Chiang. [Counting Like Transformers: Compiling Temporal Counting Logic into Softmax Transformers](#). arXiv:2404.04393, 2024.
- [4] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. [Transformers Learn Shortcuts to Automata](#). ICLR, 2023.
- [5] Shivam Garg, Dimitris Tsipras, Percy Liang, and Gregory Valiant. [What Can Transformers Learn In-Context? A Case Study of Simple Function Classes](#). NeurIPS, 2022.
- [6] Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. [Transformers Learn In-Context by Gradient Descent](#). ICML, 2023.
- [7] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. [What Learning Algorithm Is In-Context Learning? Investigations with Linear Models](#). ICLR, 2023.
- [8] Deqing Fu, Tian-Qi Chen, Robin Jia, and Vatsal Sharan. [Transformers Learn to Achieve Second-Order Convergence Rates for In-Context Linear Regression](#). NeurIPS, 2024.
- [9] Joshua Hill, Benjamin Eyre, and Elliot Creager. [Transformers Don’t In-Context Learn Least Squares Regression](#). ICML Workshop on Reliable and Responsible Foundation Models, 2025.
- [10] Ruiqi Zhang, Spencer Frei, and Peter L. Bartlett. [Trained Transformers Learn Linear Models In-Context](#). Journal of Machine Learning Research, 25(49):1–55, 2024.
- [11] Xinting Huang, Andy Yang, Satwik Bhattamishra, Yash Sarrof, Andreas Krebs, Hattie Zhou, Preetum Nakkiran, and Michael Hahn. [A Formal Framework for Understanding Length Generalization in Transformers](#). ICLR, 2025.